8

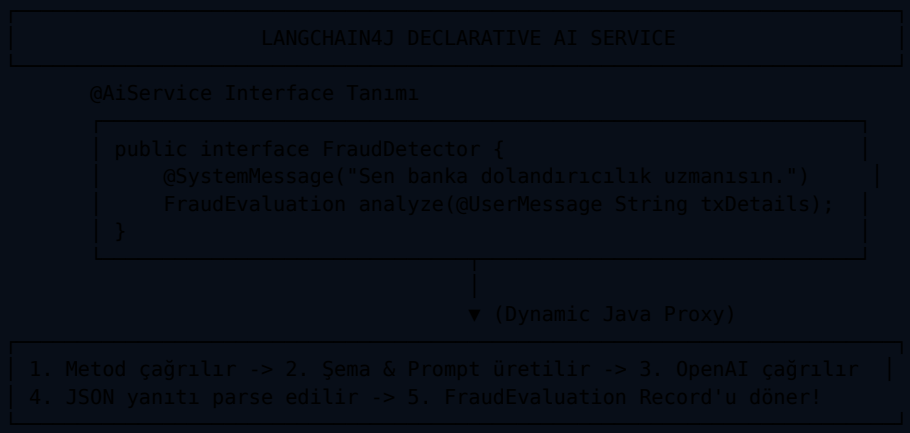
BÖLÜM: SPRING AI & LANGCHAIN4J TEMEL YAPI TAŞLARI VE CHAT MODELLERI | MODULE: CORE FOUNDATIONS: SPRING AI, LANGCHAIN4J & MODERN CHAT MODELS

LangChain4j Mimarisi: OpenAiChatModel, AiServices ve Deklaratif Tasarım

LangChain4j Architecture: OpenAiChatModel, AiServices & Declarative Design

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Spring Data JPA kullandıysanız bilirsiniz: Sadece bir `interface` yazarsınız (`UserRepository`), arkada SQL kodlarını Spring halleder. LangChain4j'in `AiServices` yapısı da tam olarak böyledir! Siz sadece bir Java arayüzü yazıp üstüne `\@SystemMessage` eklersiniz; LangChain4j sizin yerinize OpenAI'a bağlanır, doğru formatta prompt'u atar, gelen cevabı çözer ve elinize hazır bir Java nesnesi olarak teslim eder. Tek satır HTTP kodu yazmazsınız!

🇬🇧 ENGLISH (ELI5)

If you've ever used Spring Data JPA or Retrofit, you know the magic: define an interface, and the framework auto-synthesizes the implementation. LangChain4j's `AiServices` brings this exact declarative paradigm to LLMs! You define a Java interface annotated with `\@SystemMessage` and `\@UserMessage`. LangChain4j builds a dynamic proxy that serializes prompts, calls OpenAI, enforces structured JSON output schemas, and deserializes responses into your domain records automatically!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

LangChain4j Deklaratif Mekanizması: 1) Dynamic Proxy: `AiServices.builder(MyService.class)` çağrısı yapıldığında Java Dynamic Proxy ile metod çağrıları dinlenir. 2) `\@V` ve `\@UserMessage`: Metod argümanlarını prompt şablonuna bağlar. 3) Output Parsing: Metod dönüş tipi `String` dışında bir POJO/Record ise, LangChain4j OpenAI'a JSON şemasını gönderir (`response_format: json_object` veya `json_schema`). 4) TokenUsage: İstenirse `Result\<T>` sarmalayıcısı ile harcanan girdi/çıktı token sayıları doğrudan elde edilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Mechanics of LangChain4j AiServices: 1) Reflection & Proxying: The `AiServices.builder()` dynamically generates runtime proxies implementing target interfaces. 2) Prompt Interpolation: Arguments annotated with `\@V` populate template placeholders seamlessly. 3) Structured Output Protocol: If the return type is a typed Record, LangChain4j automatically injects JSON Schema instructions into the payload or leverages OpenAI's native Structured Outputs API. 4) Token Telemetry: Wrapping return types in `Result\<T>` exposes exact input/output token counts and finish reasons.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & LangChain4j: Deklaratif AiService Tanımı ve Çalıştırılması
import dev.langchain4j.service.AiServices;
import dev.langchain4j.service.SystemMessage;
import dev.langchain4j.service.UserMessage;
import dev.langchain4j.model.openai.OpenAiChatModel;

public class FraudDetectionModule {
    public record FraudEvaluation(boolean isSuspicious, int riskScore, String reason) {}

    public interface FraudService {
        @SystemMessage("Sen kıdemli bir ödeme güvenliği denetçisisin. Şüpheli işlemleri tespit et.")
        FraudEvaluation evaluateTransaction(@UserMessage String transactionDetails);
    }

    public static FraudService create(String apiKey) {
        var model = OpenAiChatModel.builder()
            .apiKey(apiKey)
            .modelName("gpt-4o-mini")
            .temperature(0.0)
            .build();

        return AiServices.create(FraudService.class, model);
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Dikkat: `AiServices` arayüzlerinde dönüş tipini karmaşık iç içe koleksiyonlar yaparsanız (ör. `Map\<String, List\<Object>>`), model JSON şemasını şaşırabilir. Daima açık ve net alanlara sahip güçlü tipli Java `record` sınıfları tercih edin.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Schema gotcha: Avoid loosely-typed return structures like `Map\<String, Object>` in AiServices interfaces. LLMs generate significantly more reliable JSON when bound to explicit, strongly-typed Java `record` declarations.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• AiServices, karmaşık LLM kodlarını tek bir temiz Java arayüzüne dönüştürür.

• Dinamik proxy yapısı, prompt oluşturma ve JSON ayrıştırmayı otomatikleştirir.

• LangChain4j, framework bağımsız mimariler ve mikroservisler için idealdir.

🎯 Key Takeaways for the Interview

• AiServices abstracts LLM plumbing behind clean, declarative Java interfaces.

• Dynamic runtime proxies automate prompt hydration and JSON deserialization.

• LangChain4j is lightweight and framework-agnostic, suitable for any JVM project.